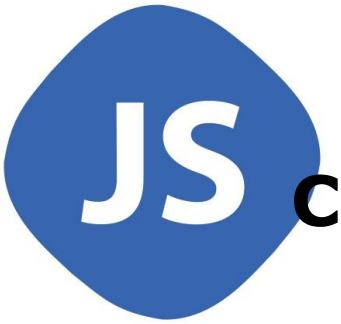


Callback функции, часть 2

- Введение
- Callback и setInterval
- Немного об асинхронности JS
- Callback в методах массива
- Callback в объекте JSON
- Callback в объекте RegExp

Введение

Callback-функции чрезвычайно важны в языке JavaScript. Они есть практически повсюду. Чтобы понимать **callback**-функции, нужно понимать **обычные функции** ().

Функции в JavaScript — немного странная конструкция, функции в Javascript — на самом деле объекты. А именно, **объекты** класса **Function**, создаваемые конструктором Function. В объекте Function содержится строка с JS-кодом данной функции (объекты – отдельная тема курса).

> ***Примечание***. Преимущество концепции функция-как-объект заключается в том, что **исходный код функции** можно **передавать в другую функцию** точно так же, как обычную **переменную** или **объект**.

Коллбэки мощное средство программирования при реализации **асинхронных** приложений. Вернее – ключевое...

Асинхронный JavaScript относится к парадигме программирования, в которой выполнение кода не следует обычному последовательному потоку сверху вниз. Вместо этого асинхронный код позволяет инициировать и выполнять определенные операции **отдельно**, **не блокируя выполнение другого кода**.



Концепция асинхронного JavaScript позволяет разбивать большие сложные проекты на более мелкие задачи.

Именно поэтому рассмотрим поведение коллбэков более подробно на основе ранее изученных технологий JavaScript. Будет интересно, и в случае с объектами RegExp – немного сложно ;)

> ***Примечание***. Для полного понимания материала необходимо повторить следующие уроки курса:
>
> Тема: **Операции в JavaScript:**
>
> - Урок 6. **JSON представление данных**
> - Урок 7. **Объект RegExp. Основы**
> - Урок 8. **Объект RegExp. Метасимволы**
>
> Тема: **Массивы и коллекции:**
>
> - Урок 2. **Свойства и методы массивов, часть 1**
> - Урок 3. **Свойства и методы массивов, часть 2**

Callback и setInterval

В прошлом уроке рассмотрели проблему асинхронного получения и обработки некоторых данных условного сервера. Вспомним код, с которого мы начинали изучение коллбэк еще раз. Он нам пригодится.

example_1. Асинхронность и данные

```
<script>

    // функция получения данных

    function getData() {

        // моделируем запрос на внешний ресурс

        setTimeout(() => {
```

```
        // определяем данные объекта user
        user = { name: "John", login: "master" }
    }, 500);
};
// функция вывода данных
function outData() {
    // если данные не определены
    if (user.name == undefined || user.login == undefined)
        // формируем предупреждение
        html = '<h2>Сбой авторизации в системе</h2>';
    else
        // иначе интерполируем переменные в строку
        html = `
            <h2>Здравствуйте, вы вошли в приложение как:</h2>
            Имя: <b>${user.name}</b> <br>
            Логин: <b>${user.login}</b> <br>
        `;
    // вывод данных в браузер
    document.getElementById("data").innerHTML = html;
};
// инициируем пустой объект
user = {};
// данные внешнего источника пришли через 0.5 сек.
getData();
// так как outData вызывается сразу, внешние данные не видны
outData();
```

```
</script>
```

Можно ли решить проблему без внедрения в функцию **getData**() пользовательской коллбэк-функции? Рассмотрим для этих целей метод **setInterval**().

Метод **setInterval**() неоднократно вызывает функцию или выполняет фрагмент кода с фиксированной задержкой между каждым вызовом. Метод возвращает **идентификатор** интервала, который однозначно идентифицирует данный интервал. Используя сохраненный идентификатор, метод **setInterval**() можно удалить вызвав **clearInterval**(). Теперь подробнее и в примерах.

setInterval

Синтаксис

```
setInterval(code);

setInterval(code, delay);

setInterval(func);

setInterval(func, delay);

setInterval(func, delay, [arg1], [arg2], ...);
```

Параметры

- **code** | **func** – строка кода или функция для выполнения. По историческим причинам рекомендуется передавать фкнуцию.
- **delay** – время в миллисекундах (тысячные доли секунды), в течение которого таймер должен задерживать выполнение указанной функции или кода. Значение по умолчанию – 0 (1000 мс = 1 с).
- **arg1**, **arg2** – дополнительные аргументы, которые передаются функции, указанной в **func**, по истечении времени таймера.

Например, следующий код будет вызывать функцию **sayHi**() с интервалом в одну секунду:

```
// определяем отложенную функцию
function sayHi() {
    // вывод в консоль браузера
    console.log('Привет!');
}
// запускаем функцию на выполнение с интервалом 1 сек.
setInterval(sayHi, 1000);
```

Возвращаемое значение

Возвращаемое значение **intervalID**— числовое, ненулевое значение, которое идентифицирует таймер, созданный вызовом **setInterval**(), это значение можно передать **clearInterval**() для отмены интервала выполнения.

Полезно знать, что **setInterval**() и **setTimeout**() используют один и тот же пул идентификаторов и что технически **clearInterval**() и **clearTimeout**() можно использовать как взаимозаменяемые.

clearInterval

Синтаксис

```
clearInterval(intervalID);
```

Параметры

- **intervalID** – идентификатор повторяющегося действия, которое вы хотите отменить. Этот идентификатор был возвращен соответствующим вызовом **setInterval**().

Возвращаемое значение – Нет.

example_2. Тестируем метод setInterval()

```
<script>
```

```
    // функция setInterval возвращает идентификатор таймера
    let intervalId = setInterval(() => {
        // получаем ссылки на узлы
        const attElem = document.getElementById("att");
        const msgElem = document.getElementById("msg");
        // управляем классом блока
        msgElem.className = msgElem.className === "blur" ? "show"
        : "blur";
        // управляем видимостью блока
        attElem.style.visibility = attElem.style.visibility ===
        "hidden" ? "visible" : "hidden";
    }, 1000);
    console.log("идентификатор setInterval: ", intervalId);
</script>
```

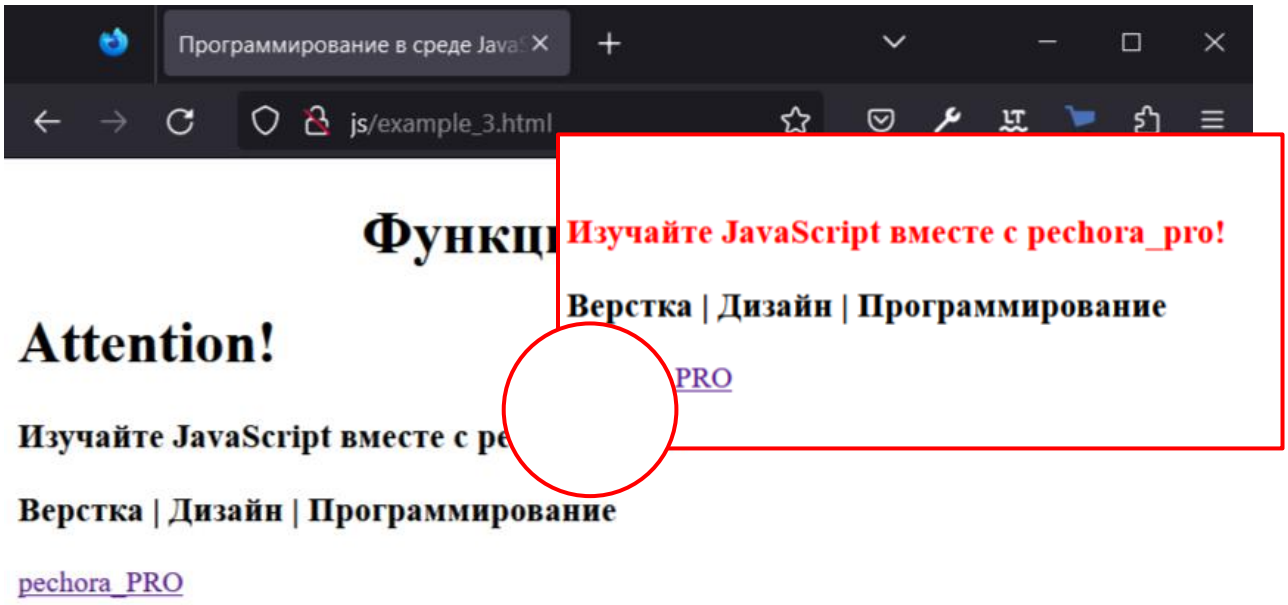
Строго говоря, мы можем эмулировать **setInterval**() с помощью рекурсивного вызова функции **setTimeout**().

example_3. Рекурсивный вызов setTimeout()

```
<script>
    // метод setTimeout с рекурсией подобен setInterval
    setTimeout(callback, 1000);
    // определение функции обратного вызова
    function callback() {
        // получаем ссылки на узлы
        const attElem = document.getElementById("att");
        const msgElem = document.getElementById("msg");
        // управляем классом блока
```

```
        msgElem.className = msgElem.className === "blur" ? "show"
        : "blur";
        // управляем видимостью блока
        attElem.style.visibility = attElem.style.visibility ===
        "hidden" ? "visible" : "hidden";
        // рекурсивно запускаем функцию callback
        setTimeout(callback, 1000);
    }
</script>
```

И в **example_2** и в **example_3** получим управляемые по видимости и классу HTML-блоки.



Мелькание блоков это забавно, но наша цель получение данных.

Немного об асинхронности JS

В следующем демонстрационном примере функция **getData**() определит данные объекта **user** через 5 секунд. В свою очередь, функция **outData**()

каждые полсекунды будет проверять наличие данных и остановит вызовы, если данные получены.

example_4. Асинхронное отслеживание данных

```
<script>
    // функция получения данных
    function getData() {
        // моделируем запрос на внешний ресурс
        setTimeout(() => {
            // определяем данные объекта user
            user = { name: "John", login: "master" }
        }, 5000);
    };
    // функция вывода данных
    function outData() {
        // функция setInterval возвращает идентификатор таймера
        intervalId = setInterval(() => {
            // если данные не определены
            if (user.name == undefined || user.login ==
            undefined) {
                // формируем предупреждение
                html = '<h2>Нет данных</h2>';
                // вывод номера запроса
                console.log(`Запрос ${k++}. Нет данных`);
            }
            else {
                // иначе интерполируем переменные в строку
```

```
                html = `
                <h2>Сервер вернул следующие данные:</h2>
                Имя: <b>${user.name}</b> <br>
                Логин: <b>${user.login}</b> <br>
                `;
                // если данные пришли, останавливаем вызовы
                clearInterval(intervalId);
            }
            // вывод данных в браузер
            document.getElementById("data").innerHTML = html;
        }, 500)
    };
    // инициируем пустой объект
    user = {};
    // счетчик количества запросов
    k = 1;
    // вызов функции получения данных
    getData();
    // вызов функции выода данных
    outData();
</script>
```

При тестировании кода демонстрационного примера **example_4** обратите внимание на нумерацию запросов, выводимых функцией **outData**(). На скриншоте рис.1 представлены асинхронные вызовы функции в процессе ожидания данных.

При желании попробуйте поменять временной интервал вызова **любой** из функций. Отследите произошедшие изменения.

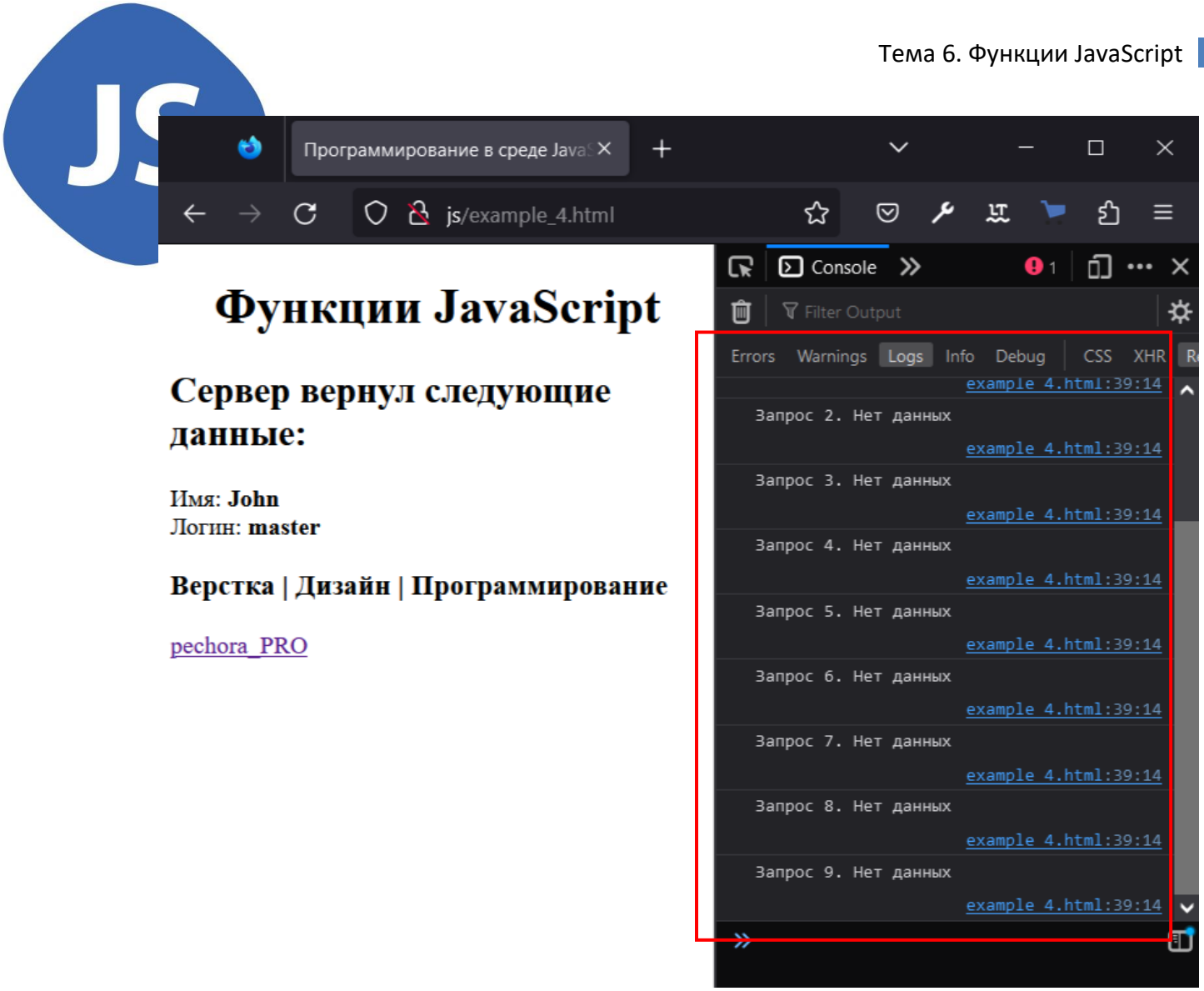


Рис. 1. Процесс получения асинхронных данных

В следующем демо примере не станем останавливать вызовы при получении данных. Сделаем процесс получения и анализа (возможного) данных постоянным.

example_5. Асинхронно работающее приложение

```
<script>

    // функция получения данных

    function getData() {

        // моделируем запрос на внешний ресурс

        setInterval(() => {
```

```
        // генерируем случайное число
        i = Math.floor(Math.random() * 5);
        // определение данных
        // используем переменную i как индекс массива
        user = persons[i];
    }, 5000);
};
// функция вывода данных
function outData() {
    intervalId = setInterval(() => {
        // если данные не определены
        if (user.name == undefined || user.surname ==
        undefined)
            // формируем предупреждение
            html = '<h2>Нет данных</h2>';
        else {
            // иначе интерполируем переменные в строку
            html = `
            <h2>Сервер вернул следующие данные:</h2>
            Имя: <b>${user.name}</b> <br>
            Логин: <b>${user.surname}</b> <br>
            `;
            console.log(html);
        }
        // вывод данных в браузер
        document.getElementById("data").innerHTML = html;
```

```
    }, 500)
};
// инициируем пустой объект
user = {};
// вызов функции получения данных
getData();
// вызов функции выода данных
outData();
</script>
```

Таким образом. Мы получили **асинхронно** работающее приложение. В таком приложении не хватает обращений к серверу, но это немного другая история (и тема курса).

Callback в методах массива

Используя коллбэк можно значительно увеличить потенциал уже знакомых нам ранее конструкций. Повторяем старый материал, изучаем новый. Начнем.

forEach

Метод **forEach(callback)** используется для перебора массива, когда для каждого элемента массива вызывается функция обратного вызова **callback(item, i, arr)** с параметрами:

- **item** – очередной элемент массива.
- **i** – его номер.
- **arr** – массив, который перебирается.

example_6. Безымянный коллбэк в теле цикла

```
<script>
    // инициализируем массив
    var course = [
        "Дизайнер интерьеров",
        "Веб-дизайнер",
        "Notion-дизайнер",
        "UX/UI-дизайнер",
        "Ландшафтный дизайнер",
        "Fashion-дизайнер"
    ];
    // перебираем элементы массива в функции коллбэк
    course.forEach(function(el, i, course) {
        console.log(++i + ". " + el);
    });
</script>
```

example_7. Внешний коллбэк для forEach

```
<script>
    // исходный массив объектов
    let teams = new Array(
        {id:1, name:"Aerosmith", style:["хард-рок", "блюз-рок",
        "рок-н-ролл", "глэм-метал"]},
        {id:2, name:"Pink Floyd", style:["психоделический-рок",
        "арт-рок", "эйсид-рок", "прогрессивный-рок"]},
        // ...
```

```
    );
    // определение функции выносим за цикл forEach
    // стрелочная функция должна быть определена до вызова
    let callback = item => {
        // сериализуем в строку свойство объекта style
        item.style = JSON.stringify(item.style);
    };
    // исходный код функции
    // console.log(callback);
    // перебираем элементы массива
    teams.forEach (callback);
    // вывод результата
    console.log(teams);
</script>
```

Метод **forEach** ничего не возвращает, его используют только для перебора, как более элегантный вариант, чем обычный цикл **for**.

filter

Метод **filter(callback)** используется для фильтрации массива через функцию обратного вызова. Создаёт **новый массив**, в который войдут только те элементы исходного массива, для которых вызов **callback(item, i, arr)** возвратит **true** (конструкция **return** функции коллбэк).

example_8. Фильтрация массива функцией коллбэк

```
<script>
    // массив скалярных значений
    var course = [
```

```
        "Дизайнер визуальных коммуникаций",
        "Ювелирный 3D-модельер",
        "Fashion-стилист",
        "Fashion-фотограф",
        "BIM-координатор с нуля",
        "Маркетинг для fashion-брендов",
        "Продюсер в Fashion"
    ];
    // создаем новый массив
    var fashionCourse = course.filter(callback);
    // определение функции обратного вызова
    // определение функции-инструкции можно выполнить после
    вызова
    function callback(el) {
        // проверяем наличие в элементе массива вхождения
        подстроки
        reg = RegExp(/fashion/, "i");
        return reg.test(el);
    }
    // проверяем элементы нового массива
    console.dir(fashionCourse);
</script>
```

Обратите внимание, метод **forEach**() просто перебирает исходный массив, тогда как **filter**() формирует новый массив, но структура массива при этом остается **прежней**. Метод **filter**() возвращает или нет текущий элемент исходного массива в зависимости от значения конструкции **return**.



В коде демонстрационного примера **example_9** тернарный оператор возвращает **true** и тогда текущий элемент попадает в новый массив, или **false**.

example_9. Конструкция return цикла filter

```
<script>
    // исходный массив
    let courses = [
        {id: 1, name: "JavaScript разработчик с нуля" , time: "7 месяцев", shedule: "По будним дням 13:00-15:00", level: "Junior", teacher: "Данилов В.Е.", price: 40000},
        {id: 2, name: "Python-рузработчик", time: "12 месяцев", shedule: "По будним дням 12:00-15:00", level: "Middle", teacher: "Ливанов Б.В.", price: 55000},
        // ...
    ];
    // функция коллбэк
    let fnCallback = (item) => {
        // элемент массива возвращается при предикате равном true
        return item.name.includes(criterion) ? true : false;
    }
    // критерий фильтрации
    const criterion = "JavaScript";
    // находит вхождения
    // или возвращает пустой массив - []
    let courseJS = courses.filter (fnCallback);
    console.log("Исходный массив:");
    console.log(courses);
    console.log("Результат фильтрации:");
```

```
    console.log(courseJS);
</script>
```

Для трансформации массива используйте цикл **map**.

map

Метод **map(callback)** используется для трансформации массива. Создаёт **новый массив**, который будет состоять из результатов вызова **callback(item, i, arr)** для каждого элемента **arr**.

example_10. Трансформация массива

```
<script>
    // преобразуем массив cars
    var dieselCars = cars.map(callback);
    // функцию инструкцию можно объявить после вызова
    function callback(item, i, cars) {
        // критерий поиска - дизельный двигатель
        reg = RegExp(/дизель/, "i");
        // если предикат true - возвращаем название
        // иначе - null
        if (reg.test(item[3])) return  item[2];
        else return null;
    }
    // вывод исходного массива
    console.dir(cars);
    // вывод дизельных авто из массива
    console.dir(dieselCars);
</script>
```

Примечание. Протестируйте отличия рассмотренных циклов. Иногда комбинация из нескольких решений даст необходимый результат

every/some

Эти методы используются для проверки массива.

- Метод **every(callback)** возвращает **true**, если вызов callback вернёт **true для каждого элемента arr**.
- Метод **some(callback)** возвращает **true**, если вызов callback вернёт **true для какого-нибудь элемента arr**.

example_11. Тестируем метод every

```
<script>
    // определяем коллбэк функции
    // все ли телевизоры имеют частоту более 60 Гц
    function checkFreq(item, i, televisions) {
        if (parseInt(item[2]) >= 60)
        return true;
    }
    // все ли телевизоры имеют LED Матрицу
    function checkMatrix(item, i) {
        if ( /led|LED/.test(item[1]))
        return true;
    }
    // есть ли телевизоры дороже 200.000
    function checkPrice(item) {
        if (parseFloat(item[6]) > 70)
```

```
        return true;
    }
    console.log("Тестируем -Every");
    console.log("-> все ли телевизоры имеют частоту более 60 Гц
    ?");
    console.log(televisions.every(checkFreq));
    console.log("-> все ли телевизоры имеют LED Матрицу ?");
    console.log(televisions.every(checkMatrix));
    console.log("-> есть ли телевизоры дороже 200.000 ?");
    console.log(televisions.every(checkPrice));
</script>
```

example_12. Тестируем метод some

```
<script>
    // определяем коллбэк функции
    // все ли телевизоры имеют частоту более 60 Гц
    function checkFreq(item) {
        if (parseInt(item[2]) >= 60)
        return true;
    }
    // все ли телевизоры имеют LED Матрицу
    function checkMatrix(item, i) {
        if (/led|LED/.test(item[1]))
        return true;
    }
    // есть ли телевизоры дороже 200.000
```

```
    function checkPrice(item, i, televisions) {
        if (parseFloat(item[6]) > 70)
        return true;
    }
    console.log("Тестируем -Some");
    console.log("-> все ли телевизоры имеют частоту более 60 Гц
    ?");
    console.log(televisions.some(checkFreq));
    console.log("-> все ли телевизоры имеют LED Матрицу ?");
    console.log(televisions.some(checkMatrix));
    console.log("-> есть ли телевизоры дороже 200.000 ?");
    console.log(televisions.some(checkPrice));
</script>
```

Callback в объекте JSON

JSON.stringify

Метод **stringify**() преобразует объект JS в строку JSON.

```
let json = JSON.stringify(value, [replacer, space]);
```

Параметры

- **value** – значение для кодирования.
- **replacer** – массив свойств для кодирования или **функция соответствия**, которая **будет вызываться для каждой пары (key, value)**.
- **space** – дополнительное пространство (отступы), используемое для форматирования.





Возвращаемое значение

JSON-строка.

В большинстве случаев **stringify** используется только с **первым** аргументом. Но если нужно настроить процесс преобразования, то можно использовать второй аргумент. Второй аргумент **либо массив** свойств для кодирования, **либо функция** соответствия.

example_13. Функция callback определяет правила преобразования

```
<script>
    // определение исходного JS-объекта
    let person = {
        surname : "Бородина",
        name : "Ксения",
        patronymic : "Алексеевна",
        category : "Соответствие занимаемой должности",
        post : "Преподаватель",
        level_edu : "Высшее профессиональное",
        email : "borodina-x@gmail.com",
        phone : "2-12-85-0A",
        experience_college : 14.5
    };
    // сериализуем JSON с дополнительной функцией
    personJSON = JSON.stringify(person, fnCollback);
    // вывод сериализованного JSON
    console.log(personJSON);
    // функция преобразования
    function fnCollback (key, value) {
```

```
        if (key == "experience_college") return undefined;
        if (key == "email" ) return "metodic@pedsov.ru";
        if (key == "phone") return "Скрыто пользователем";
        // по умолчанию возвращаем исходное значение - value
        return value;
    }
</script>
```

JSON.parse

Чтобы декодировать JSON-строку нужен метод **parse**(). Метод преобразует строку JSON в соответствующее значение JavaScript.

Если во время преобразования в строке JSON встретится значение **undefined**, то оно будет опущено (не будет включено в результат).

```
let value = JSON.parse(str, [reviver]);
```

Параметры

- **str** - JSON для преобразования в объект.
- **reviver** - необязательная **функция**, которая будет **вызываться для каждой пары (key, value)** и может преобразовывать значение.

example_14. Парсим строку в объект с дополнительной функцией

```
<script>
    // определение JSON-строки
    let personJSON = `{
        "surname" : "Бородина",
        "name" : "Ксения",
        "post" : "Преподаватель",
```

```
        "email" : "borodina-x@gmail.com",
        "phone" : "2-12-85-0A",
        "experience_college" : 14.5
    }`;
    // парсим JSON с дополнительной функцией
    // коллбэк встроен в метод parse
    let person = JSON.parse(personJSON, function (key, value) {
        if (key == "experience_college") return undefined;
        if (key=="phone") return "*****";
        if (key == "email" ) return "metodic@pedsov.ru";
        // по умолчанию возвращаем исходное значение - value
        return value;
    });
    // вывод в консоль результата преобразования
    console.log(person);
</script>
```

В примере правило сериализации определим через функцию соответствия **function(key, value)**. Эта функция вызывается для каждой пары **key**, **value** и может менять правила преобразования.

Callback в объекте RegExp

Коллбэк (callback) в регулярных выражениях (RegExp) дает неограниченные возможности управления DOM-документа. Но есть важный момент.

Примечание. Возможности регулярных выражений и функций коллбэк в них настолько велики, что многие программисты выносят регулярные выражения

в отдельную ветвь программирования.

Первый пример преобразования с дополнительной функцией достаточно простой. Функция callback получает на вход результат поиска с использованием шаблона:

```
// глобальный шаблон поиска
let reg = new RegExp("[a-z]{3,}", "ig");
// метод replace вызовет callback для каждой найденной с помощью шаблона строки
let edit_text = text.replace(reg, callback); // вызов callback
```

Разбираемся, дальше будет сложнее.

example_15. Преобразование с помощью коллбэк RegExp

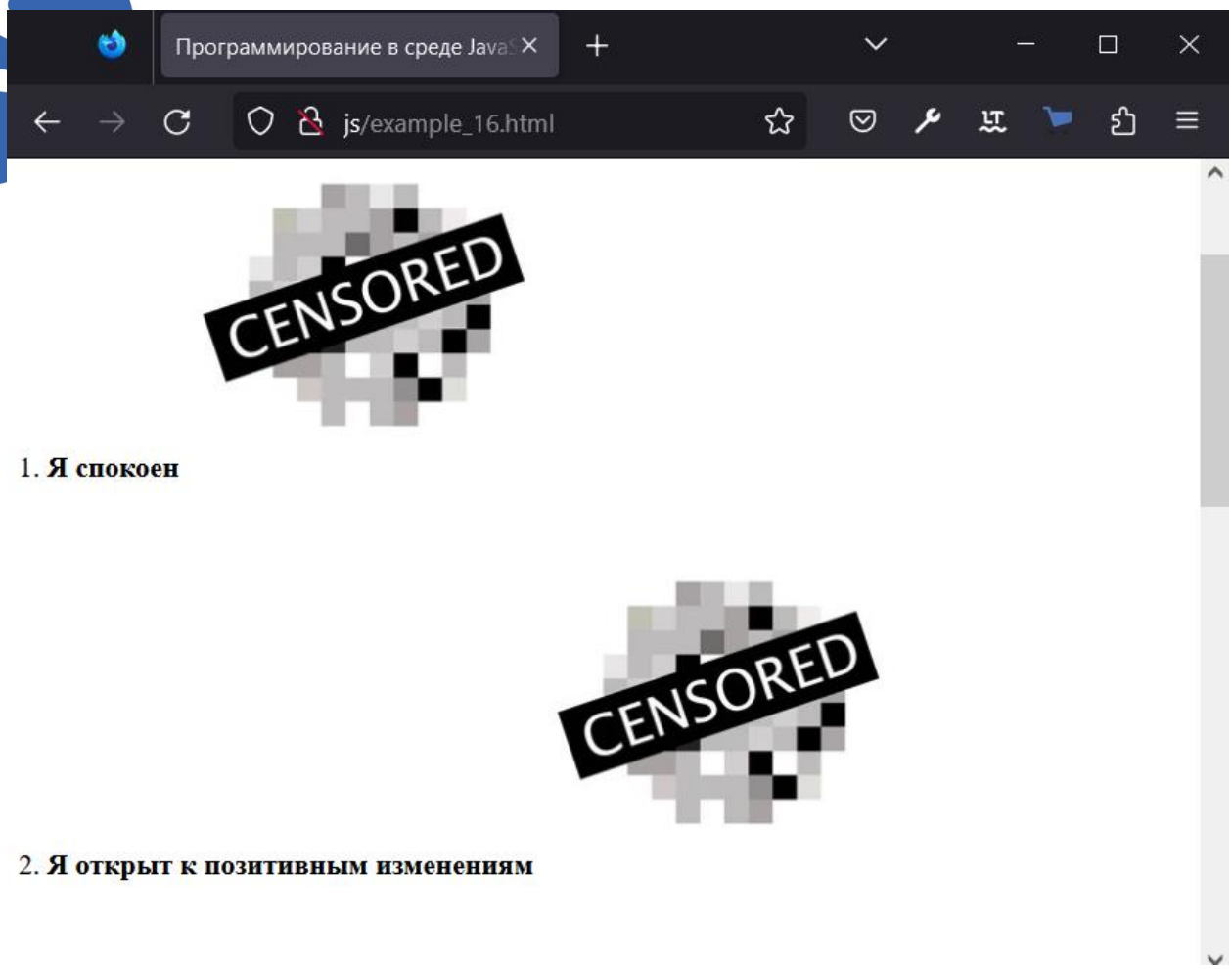
```
<script>
    // строка в которой будем искать совпадение с шаблоном
    let text = document.querySelector("div#text").innerHTML;
    callback = (elem) => {
        elem = "<b>" + elem + "</b>";
        // возвращаем преобразованный элемент
        return elem;
    }
    // глобальный шаблон поиска
    let reg = new RegExp("[a-z]{3,}", "ig");
    // поиск и замена
    let edit_text = text.replace(reg, callback);
    // вставляем в документ измененный текст
    document.querySelector("div#edit").innerHTML = edit_text;
```

```
</script>
```

example_16. Усложняем код callback для RegExp

```
<script>
    // функция коллбэк
    fnCallback = (el) => {
        // console.log(el);
        // match с флагом -g и без него дадут разный результат
        let info = el.match(/pict[1-5]\.jpg/);
        // console.log(info);
        return `<img src="censored.jpg" title="Исходное
        изображение: ${info[0]}" width="200px">`;
    }
    // забираем из документа HTML-содержимое блока div
    var str = document.body.querySelector("div").innerHTML;
    // получаем обновленый текстовый контент
    var newstr = str.replace(/<img .*>/g, fnCallback);
    // замещаем блок div новым контентом
    document.body.querySelector("div").innerHTML = newstr;
</script>
```

В демонстрационном примере **example_16** мы могли насладиться видом красивых картинок *мультяшных* героев. Но код примера для регулярного выражения не позволяет этого сделать, а значит метод **replace** и функция **fnCallback** работают, а это хорошо...



Следующий пример – начало эксперимента по управлению контентом страницы документа.

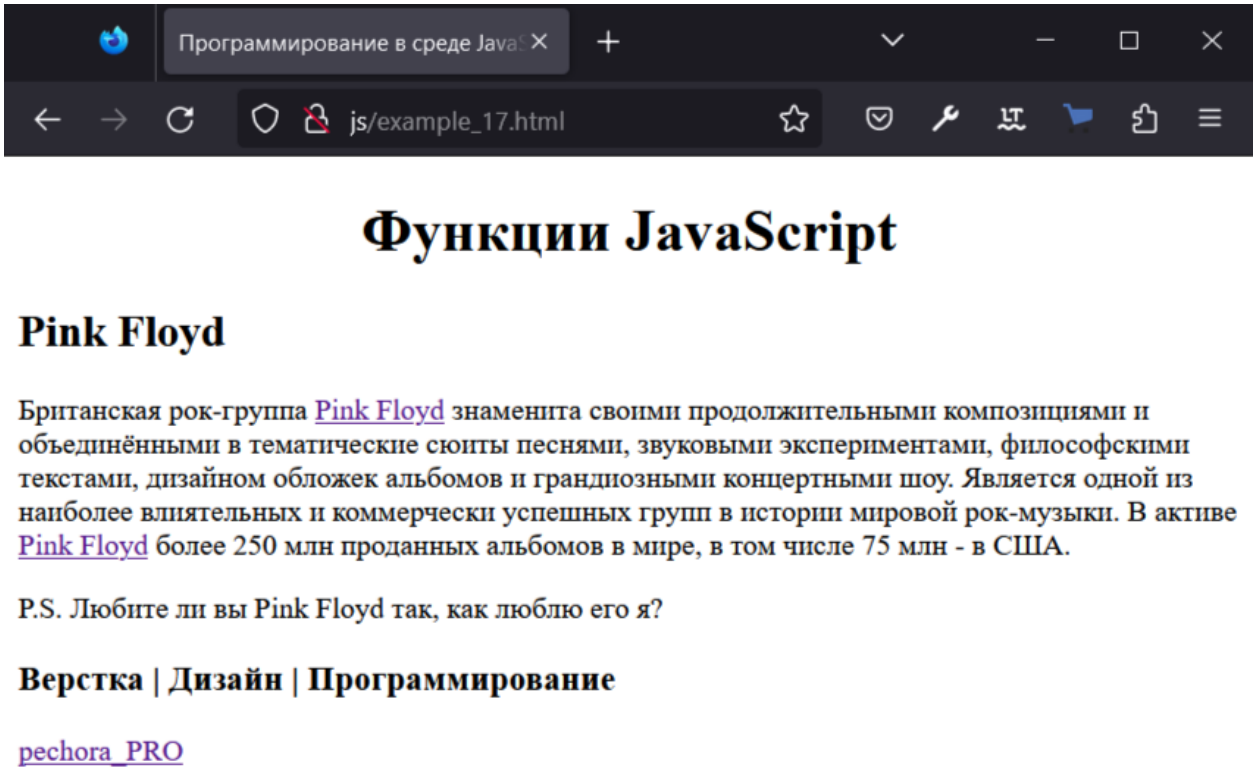
example_17. Вставка ссылок в текстовый контент

```
<script>
    // забираем из документа содержимое блока div
    var str = document.body.querySelector("div").innerHTML;
    callback = (el) => {
        // console.log(el);
        let link = "https://ru.wikipedia.org/wiki/Pink_Floyd";
        let match = el.match(/{{([\s-_a-z]{1,20}[^\s])\s?}}/i);
        // console.log(match);
        return `<a href="${link}" target="_blank"> ${match[1]}
```

```
        </a>`;
    }

    // забираем из документа содержимое блока div
    var str = document.body.querySelector("div").innerHTML;
    // выполняем замену
    var newstr = str.replace(/{{[a-z\s-]{1,50}}}/ig, callback);
    // замещаем блок новым содержимым
    document.body.querySelector("div").innerHTML = newstr;
</script>
```

Результат работы скрипта представлен на следующем скриншоте.



В самостоятельных уроках разовьем логику демо примера **example_17**.

Примечание. Логика записи шаблонов регулярных выражений самостоятельных работ представлена из соображений наилучшего

восприятия, а не оптимального выполнения. Экспериментируйте. Нормальной практикой является предварительное ознакомление с предлагаемым решением самостоятельной работы.

Закрепляем пройденный материал. В **самостоятельных работах** выполним следующие задания:

- Поиск объекта в асинхронных вызовах.
- Создание коллбэк функции для поиска элемента в значениях массива.
- Написание шаблонов регулярных выражений.
- Поиск в коллбэк функции регулярного выражения.

P.S.

Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:

- **Практические** работы.